

## Projeto II: Sistema de Gestão de Voluntario

**Disciplina:** Banco de Dados Orientado a Objetos (BDOO)

**Tema:** Sistema de Automação, Otimização e Segurança de Projetos Sociais

### 1. O Desafio da Equipe

O grupo foi contratado para desenvolver o **Conecta Voluntário**, um sistema robusto para o gerenciamento de engajamento social e oportunidades de voluntariado. Neste projeto, o MySQL não será apenas um depósito de dados; ele atuará como o "cérebro" que protege as regras de negócio e a segurança. O Java servirá como a interface amigável que interage com esse cérebro.

A equipe deverá aplicar todo o conteúdo visto na disciplina, garantindo que a segurança seja imposta pelo banco de dados e não apenas pela interface gráfica.

### 2. A Base: Estrutura do Banco de Dados (DDL)

Antes de escrever qualquer linha de Java, a equipe precisa garantir que o alicerce (o banco `db_conecta_voluntario`) seja sólido. Um banco mal projetado fará com que o código Java quebre ou fique complexo demais.

Vocês devem criar as tabelas abaixo respeitando as chaves (PK e FK) e os tipos de dados adequados.

#### 1. Tabela Usuarios (A entidade Pai)

Esta tabela armazenará todos os atores do sistema. Para suportar a herança no Java (Voluntario, Gestor\_ONG, Admin), usaremos uma coluna discriminadora (tipo).

- `id_usuario` (INT, PK, AUTO\_INCREMENT): A identidade única e imutável.
- `nome` (VARCHAR): Nome completo.
- `cpf` (CHAR(11), UNIQUE): Deve ser chave única para impedir duplicidade de cadastro. Usem CHAR pois tem tamanho fixo.
- `email` (VARCHAR, UNIQUE): Essencial para comunicação e login.
- `senha` (VARCHAR): Nunca armazenem senhas curtas; pensem no tamanho do hash.
- `tipo` (ENUM ou VARCHAR): Valores esperados: 'VOLUNTARIO', 'ADMIN', 'GESTOR\_ONG', 'INICIANTE'.

apagado.

## 2. Tabela endereços (Normalização – 1FN)

Seguindo a 1ª Forma Normal (evitar grupos de repetição e colunas compostas), o endereço não deve ficar na tabela de usuários.

- id\_endereco (INT, PK, AUTO\_INCREMENT);
- logradouro, bairro, cidade, uf: Campos atômicos;
- id\_usuario\_fk (INT, FK): Chave Estrangeira que liga este endereço a um usuário específico. Lembrem-se da Integridade Referencial: não se pode cadastrar um endereço para um usuário que não existe.

## 3. Tabela Oportunidades (As vagas/Ações)

- id\_oportunidade (INT, PK, AUTO\_INCREMENT).
- titulo (VARCHAR): Título da ação social.
- ong\_responsavel (VARCHAR): Na 3FN ideal seria uma tabela separada, mas para este projeto, aceitaremos nesta tabela.
- horas\_estimadas (DECIMAL(10,2)): Atenção: Nunca usem FLOAT OU DOUBLE para quantificar. O tipo DECIMAL garante a precisão de horas/frações.
- vagas\_disponiveis (INT): Controlado pelas triggers de inscrição/cancelamento.
- status (VARCHAR): Ex: 'ABERTA', 'ENCERRADA'.

## 4. Tabela Inscricoes(Tabela Associativa)

Esta tabela resolve o relacionamento "Muitos-para-Muitos" entre Usuários e Oportunidades.

- id\_inscricao (INT, PK, AUTO\_INCREMENT).
- id\_usuario\_fk (INT, FK): Quem se inscreveu.
- id\_oportunidade\_fk (INT, FK): Em qual ação social se inscreveu.
- data\_inscricao (DATETIME): Data e hora exata da candidatura (Use DEFAULT CURRENT\_TIMESTAMP).
- data\_prevista\_conclusao (DATE): Quando a ação deve terminar (Calculado pelo Java ou Procedure).
- data\_conclusao\_real (DATETIME): Inicialmente NULL. Só é preenchida quando o projeto é finalizado pelo voluntário.

apagado.

## 5. Tabela Impacto\_Horas (O Resultado Financeiro/Horas)

- id\_impacto (INT, PK, AUTO\_INCREMENT).
- id\_inscricao\_fk (INT, FK): Vincula o impacto gerado a uma inscrição específica.
- horas\_validadas (DECIMAL(10,2)): Valor calculado pela procedure de validação de projeto.
- certificado\_gerado (BOOLEAN ou TINYINT): 0 para pendente, 1 para gerado.

## 6. Tabela Log\_Auditoria (Segurança)

Esta tabela será alimentada exclusivamente pela Trigger de Auditoria.

- id\_log (INT, PK, AUTO\_INCREMENT).
- tabela\_afetada (VARCHAR): Ex: 'Oportunidades'.
- acao (VARCHAR): Ex: 'DELETE', 'UPDATE VAGAS'.
- usuario\_responsavel (VARCHAR): Quem estava logado no banco (função USER() OU CURRENT\_USER()).
- dados\_antigos (TEXT): JSON ou texto contendo o registro OLD que foi apagado.
- data\_hora (TIMESTAMP): DEFAULT CURRENT\_TIMESTAMP.

## 3. Segurança no Banco (O "Coração" do Projeto)

A equipe deve implementar o princípio do Privilégio Mínimo. Vocês criarão 4 usuários distintos no MySQL, e a aplicação Java deverá conectar-se usando essas credenciais específicas, dependendo de quem está fazendo login. Abaixo estão as regras que devem ser aplicadas via permissões (GRANT/REVOKE):

1. **usr\_admin (O Administrador):** Este usuário deve ter acesso total ao banco (ALL PRIVILEGES). A função dele é gerencial: ele é o único autorizado a ver dados sensíveis e o único capaz de executar o comando de Backup.
2. **usr\_gestor\_ong (O Operador):** É o responsável pela rotina diária. Ele precisa de permissão para consultar vagas, registrar novos projetos e atualizar status de inscrições. Porém, por segurança, ele não deve ter permissão para apagar registros da tabela de auditoria.
3. **usr\_iniciante (O Usuário Restrito):** Este perfil serve para provar a segurança do sistema. Ele deve ter permissões mínimas: apenas consultar vagas e inserir candidaturas simples. A equipe deve revogar explicitamente o direito de DELETE deste usuário. Se ele tentar apagar uma oportunidade, o banco deve bloquear.
4. **usr\_visitante (O Visitante):** Este usuário terá acesso apenas de leitura (SELECT). Para proteger dados sigilosos, ele não deve ter acesso direto às tabelas físicas. O acesso dele deve ser feito exclusivamente através de Views públicas criadas pela equipe.

## 4. Objetos Ativos

A equipe deve implementar os seguintes objetos no banco:

### 1. Quatro Stored Procedures (Procedimentos Armazenados)

- `sp_transacao_inscricao` (`id_user`, `id_oportunidade`): Inicia Transação. Verifica se o usuário tem pendências. Verifica vagas\_disponiveis. Insere na tabela Inscricoes. Decrementa 1 no estoque de vagas da tabela Oportunidades. Se qualquer passo falhar (ex: vagas zero), faz ROLLBACK; senão, COMMIT.
- `sp_renovar_prazo_acao` (`id_inscricao`): Verifica se a vaga possui status de "Encerrada". Se estiver livre, atualiza a `data_prevista_conclusao` somando 7 dias. Se houver restrição, nega a operação e retorna uma mensagem de erro.
- `sp_calcular_impacto_horas` (`id_inscricao`, `OUT total_horas`): Recebe o ID da inscrição. Compara a data de início com a data de conclusão\_real. Calcula as horas trabalhadas geradas. Retorna esse valor no parâmetro de saída `OUT` para que o Java possa exibir na tela antes de gerar o certificado.
- `sp_transacao_cadastro_completo` (`nome`, `cpf`, `rua`, `num`): Inicia Transação. Insere os dados na tabela Usuarios. Recupera o ID gerado (`LAST_INSERT_ID()`). Insere os dados na tabela Enderecos usando esse ID. Se o endereço falhar, desfaz a inserção do usuário (ROLLBACK) para não deixar registros órfãos.

### 2. Quatro Triggers (Gatilhos)

- `trg_trava_horario_comercial`: Dispara BEFORE INSERT/UPDATE em ações de ONGs. Se for fora do horário administrativo (08h-18h), aborta com erro (SIGNAL SQLSTATE).
- `trg_auditoria_delecao`: Dispara AFTER DELETE em Oportunidades. Salva os dados apagados (OLD) e o usuário na tabela de Log.
- `trg_limite_inscricoes`: Dispara BEFORE INSERT. Bloqueia se o voluntário já tiver 3 inscrições simultâneas ativas (em andamento).
- `trg_preventiva_vagas`: Dispara BEFORE UPDATE. Impede que o número de `vagas_disponiveis` fique negativo.

### 3. Quatro Views (Visões)

- `w_vagas_publicas`: Para uso do Visitante. Mostra dados da oportunidade, ocultando detalhes de contatos diretos e documentos da ONG.
- `vw_inscricoes_pendentes`: Lista candidaturas aguardando aprovação com contato do voluntário.
- `vw_ranking_impacto`: Top 10 voluntários com mais horas validadas (GROUP BY/COUNT).
- `vw_dashboard_ong`: Soma total de horas geradas nos projetos daquela organização (SUM).

## 5. A Aplicação Java: POO, Login e Menus Inteligentes

O sistema não deve utilizar o terminal da IDE para funcionar, a interface deve ser construída inteiramente utilizando a biblioteca `javax.swing.JOptionPane`, caso queira usar outro tipo de interface gráfica, comunique ao professor. O objetivo não é fazer telas bonitas, mas sim funcionais, que respeitem as regras de segurança impostas pelo banco de dados.

O fluxo de execução obrigatório é o seguinte:

### Passo 1: Conceitos de POO

O código fonte deve implementar explicitamente:

#### 1. Encapsulamento:

- Todos os atributos das classes de modelo (Usuario, Oportunidade, Inscricao) devem ser `private`;
- O acesso deve ser feito obrigatoriamente via *getters* e *setters*. Validem dados nos *setters* (ex: não permitir horas estimadas negativas).

#### 2. Herança:

- Aproveitem a coluna tipo da tabela Usuarios para criar uma hierarquia de classes;
- Classe Mãe: abstrata Usuario;
- Classes Filhas: Voluntario e GestorONG que herdam de Usuario.

#### 3. Polimorfismo:

- Implementem um método abstrato na classe mãe que se comporte de forma diferente nas subclasses.
- Sugestão: `getDiasPrazoConclusao()`. Na classe Voluntario retorna 7 dias, no GestorONG retorna 14 dias;
- O Java deve usar esse método polimórfico para calcular a data antes de enviar para o banco.

## Passo 2: Seleção de Perfil Inicial

Use `JOptionPane.showOptionDialog`. A Pergunta: "Qual é o seu perfil de acesso?".  
As Opções: ["Gestor ONG", "Voluntário", "Sair"].

- Se escolher "Voluntário" -> Vai para a tela de Login e depois carrega o Menu A.
- Se escolher "Gestor ONG" -> Vai para a tela de Login e depois carrega o Menu B.

## Passo 3: Autenticação Real (Login no Banco)

Vocês NÃO devem ter um usuário e senha fixos no código.

- O que fazer: Use `showInputDialog`;
- Pedindo: Usuário do Banco (ex: `usr_iniciante`, `usr_gestor_ong`) e Senha do Banco;
- A Conexão: Passem essas variáveis digitadas para a classe `Conexao.java`. Se o usuário digitar `usr_iniciante`, a conexão Java terá apenas as permissões restritas (REVOKE DELETE).

## Passo 4: O Menu Dinâmico

**Cenário A: Se o usuário escolheu "Voluntário"** O menu deve ser um laço (do-while) exibindo as opções:

1. Consultar Vagas Disponíveis (`SELECT * FROM vw_vagas_publicas`).
2. Minhas Inscrições (Filtra a procedure de histórico pelo ID do aluno logado).
3. Sair.

**Cenário B: Se o usuário escolheu "Gestor ONG"** O menu deve ser padronizado. Não tentem esconder botões via `if` no Java; deixem o banco barrar o acesso.

1. Cadastrar Oportunidade (Aquisição).
2. Aprovar Inscrição (Chama procedure para alocar voluntário).
3. Validar Horas Concluídas (Chama `sp_calcular_impacto_horas`).
4. Excluir Oportunidade (O Teste de Fogo): Executa `DELETE FROM oportunidades`.
5. Gerar Relatórios / Backup.
6. Sair.

## Passo 5: A Armadilha do Estagiário (Tratamento de Exceção)

Este é um requisito obrigatório. Como o menu de Gestor é único, um perfil Iniciante logado acidentalmente ali verá o botão "4. Excluir Oportunidade".

1. A Ação: O `usr_iniciante` clica em "Excluir Oportunidade".
2. O Código Java: O sistema pede o ID e chama `vagaDAO.deletar(id)`.
3. O Banco: O MySQL bloqueia a ação (REVOKE DELETE) e lança uma exceção.
4. O Resultado: No catch (`SQLException` e), o Java mostra o `JOptionPane`: "ERRO: Acesso Negado! Seu perfil de usuário não tem permissão para excluir registros do sistema."



## 6. Otimização e Infraestrutura

A equipe deve provar que o banco é performático utilizando Índices e a ferramenta EXPLAIN.

1. **Índices:** Criar 3 índices estratégicos (ex: em título na tabela de oportunidades ou cpf para login) e documentar a justificativa.
2. **Relatório EXPLAIN:** Demonstrar a melhoria "Antes e Depois" executando uma consulta pesada. O type deve mudar de ALL (lento) para REF ou CONST (rápido).

## 7. Plano de Testes

- **Teste de Horário:** Tentar inserir projeto da ONG às 22h. Deve dar erro de Trigger.
- **Teste de Atomicidade:** Usar a `sp_transacao_cadastro_completo` com endereço inválido para provar que o usuário não foi salvo (ROLLBACK).
- **Teste de Segurança:** Logar como Iniciante e tentar excluir uma Oportunidade.
- **Teste de Auditoria:** Excluir uma vaga como Admin e checar a tabela Log\_Auditoria.
- **Teste de Limite:** Tentar inscrever o voluntário em uma 4ª vaga pendente.
- **Teste de Impacto:** Simular uma finalização de projeto e verificar se a procedure retornou o cálculo de horas correto.
- **Teste de View:** Logar como Visitante e tentar ver colunas sigilosas da ONG (não deve aparecer).
- **Teste de Explain:** Rodar o comando antes e depois de criar o índice.

## 8. Entrega e Apresentação

A avaliação final será composta pela entrega dos arquivos e pela defesa do projeto em:

**A. Apresentação em Sala:** Slides com Capa, Arquitetura Java (Classes, DAO), Soluções SQL complexas, Análise EXPLAIN e Demonstração Ao Vivo executando o Plano de Testes, focando na "Armadilha do Iniciante" para provar segurança.

**B. Submissão no Canvas:** Arquivo .zip contendo Código Fonte Java (src), Script SQL de Criação (tabelas, procedures, views), Script SQL de Dados (Dump) e Slides.